

Statistics for Data Analytics – CA 2

Student Report

Name: Harsanbruno Maria Joseph Esuraj

Module: Statistics for Data Analytics (B9DA101)

Student Number: 20065575

Institution: Dublin Business School

Instructor: Dr. Muhammad Alli

Submission Type: Individual Software

Used: R (RStudio)

Date: 20-04-2025

Question 1: Generalized Linear Modelling (GLM)

(a) Training a GLM Model

The dataset used is the put-in 'infert' dataset in R. The outputs variable (case) are related to input variables (parity, age, spontaneous, induced). 80% of the dataset was used for training. The GLM was fitted for Gaussian family.

```
# Load the inbuilt dataset
infert_data <- infert
print(head(infert_data))
```

After loading the dataset

```
  education age parity induced case spontaneous stratum pooled.stratum
1    0-5yrs  26     6      1     1         2         1         3
2    0-5yrs  42     1      1     1         0         2         1
3    0-5yrs  39     6      2     1         0         3         4
4    0-5yrs  34     4      2     1         0         4         2
5    6-11yrs 35     3      1     1         1         5        32
6    6-11yrs 36     4      2     1         1         6        36
> |
```

Trained the model at 80%

```
# (a) Train the model (80% training)
set.seed(123)
train_indices <- sample(1:nrow(infert_data), size = 0.8 * nrow(infert_data))
training_set <- infert_data[train_indices, ]
testing_set <- infert_data[-train_indices, ]
|
```

Fitting the Glm model gaussian family and Printing the summary

```
# Fitting a GLM model (Gaussian family)
model_glm <- glm(case ~ parity + age + spontaneous + induced,
  data = training_set,
  family = gaussian())

# Print the summary
summary_output <- summary(model_glm)
print(summary_output)
```

After fitting and Printing the summary

```
Call:
glm(formula = case ~ parity + age + spontaneous + induced, family = gaussian(),
  data = train_data)

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.083937   0.197087   0.426 0.670662
parity      -0.125194   0.032015  -3.911 0.000127 ***
age           0.006291   0.006001   1.048 0.295860
spontaneous  0.356244   0.051681   6.893 7.55e-11 ***
induced       0.208259   0.054506   3.821 0.000179 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.1854779)

    Null deviance: 44.646  on 197  degrees of freedom
Residual deviance: 35.797  on 193  degrees of freedom
AIC: 235.24

Number of Fisher Scoring iterations: 2
```

(b) Significance of Variables at alpha=0.05

Significant variables ($p < 0.05$) were found to be parity and induced and Spontaneous

The following variables were found statistically significant with p-values less than 0.05:

Parity: $p = 0.000127$

Spontaneous: $p = 7.55e-11$

Induced: $p = 0.000179$

This result rejects the null hypothesis for these variables, showing they have a significant effect on the output variable (case).

```
# (b) Significant variables at the level of alpha = 0.05
coeff_table <- summary_output$coefficients
significant_vars <- coeff_table[coeff_table[, 4] < 0.05, ]
print("Significant Variables (p < 0.05):")
print(significant_vars)
```

	Estimate	Std. Error	t value	Pr(> t)
parity	-0.1251940	0.03201460	-3.910528	1.274739e-04
spontaneous	0.3562441	0.05168052	6.893199	7.550616e-11
induced	0.2082594	0.05450584	3.820863	1.792482e-04

```
> |
```

(c) Prediction and Structure of Model

Model formula:

$Y = \text{beta0} + \text{beta1}(\text{parity}) + \text{beta2}(\text{age}) + \text{beta3}(\text{spontaneous}) + \text{beta4}(\text{induced})$

The model was used to produce predictions on test data. It calculated RMSE.

```
# (c) Predict the output of test dataset
predictions_glm <- predict(model_glm, testing_set)
print("Predicted Values:")
print(predictions_glm)
```

Predicted values:

```
[1] "Predicted Values:"
> print(pred)
      2      3      12      15      18      19
0.431213394 -0.005369405 0.588681194 0.593065416 0.478546382 0.638386377
      27      28      31      37      47      65
0.785085286 0.634728359 0.516290786 0.541453721 0.722177946 0.205367660
      73      80      82      87      95     105
0.510000052 0.528872254 0.497418584 0.153290282 0.380421839 0.053724910
     111     114     123     132     134     144
-0.077759814 0.160046699 0.343143118 0.205367660 0.632095643 0.470243620
     150     152     156     162     163     172
0.447092712 0.505615830 0.510000052 0.172628167 0.053724910 0.103430093
     173     174     178     180     187     189
0.391096795 0.299107981 0.153755965 0.236821330 0.053724910 0.541453721
     191     192     194     200     201     213
0.085178580 0.072597112 0.532064589 0.204081837 0.053724910 0.169529768
     215     216     218     219     227     228
0.155662477 0.275851557 0.294723758 0.491127850 0.324270916 0.172628167
     230     244
0.709596478 0.510000052
```

Print the actual output:

```
# Actual values
actual_values <- testing_set$case
print("Actual Values:")
print(actual_values)
```

```
[1] "Actual Values:"
> print(act)
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
[38] 0 0 0 0 0 0 0 0 0 0 0 0 0 0
> |
```

Compare predictions vs actual values

```
# Compare predictions vs actual
comparison_df <- data.frame(Actual = actual_values, Predicted = predictions)
print("Comparison:")
print(comparison_df)
```

	Actual	Predicted			
2	1	0.431213394	162	0	0.172628167
3	1	-0.005369405	163	0	0.053724910
12	1	0.588681194	172	0	0.103430093
15	1	0.593065416	173	0	0.391096795
18	1	0.478546382	174	0	0.299107981
19	1	0.638386377	178	0	0.153755965
27	1	0.785085286	180	0	0.236821330
28	1	0.634728359	187	0	0.053724910
31	1	0.516290786	189	0	0.541453721
37	1	0.541453721	191	0	0.085178580
47	1	0.722177946	192	0	0.072597112
65	1	0.205367660	194	0	0.532064589
73	1	0.510000052	194	0	0.532064589
80	1	0.528872254	200	0	0.204081837
82	1	0.497418584	201	0	0.053724910
87	0	0.153290282	213	0	0.169529768
95	0	0.380421839	215	0	0.155662477
105	0	0.053724910	216	0	0.275851557
111	0	-0.077759814	218	0	0.294723758
114	0	0.160046699	219	0	0.491127850
123	0	0.343143118	227	0	0.324270916
132	0	0.205367660	228	0	0.172628167
134	0	0.632095643	230	0	0.709596478
144	0	0.470243620	244	0	0.510000052
150	0	0.447092712			
152	0	0.505615830			
156	0	0.510000052			

Checking Root mean square error:

```
# Calculate RMSE
rmse_value <- sqrt(mean((predictions_glm - actual_values)^2))
print(paste("RMSE:", rmse_value))
```

```
[1] "RMSE: 0.404359496337955"
```

(d) Classification and Accuracy

Predictions were binarized at the 0.5 cut-off. A confusion matrix and accuracy were computed.

```
# (d) Classification using 0.5 threshold
predicted_class <- ifelse(pred > 0.5, 1, 0)
actual_class <- test_data$case

# Confusion Matrix
conf_matrix <- table(Predicted = predicted_class, Actual = actual_class)
print("Confusion Matrix:")
print(conf_matrix)

# Accuracy
accuracy <- sum(predicted_class == actual_class) / length(actual_class)
print(paste("Accuracy:", accuracy))
```

Accuracy and confusion matrix

```

      Actual
Predicted 0  1
0      28  5
1       7 10
>
> # Accuracy
> accuracy <- sum(predicted_class == actual_class) / length(actual_class)
> print(paste("Accuracy:", accuracy))
[1] "Accuracy: 0.76"
```

Question 2: Bayesian Inference with Poisson Likelihood

(a) Likelihood Function

Given $X_i \sim \text{Poisson}(\lambda)$, the probability mass function is

$$P(X_i = x_i) = \frac{e^{-\lambda} \lambda^{x_i}}{x_i!}$$

For i.i.d data, the likelihood function is:

$$L(\lambda) = \prod_{i=1}^n \frac{e^{-\lambda} \lambda^{x_i}}{x_i!} = \frac{e^{-n\lambda} \lambda^{\sum_{i=1}^n x_i}}{\prod_{i=1}^n x_i!}$$

Given $X = \{1, 3, 2, 2, 0, 3, 4, 3, 2, 1\}$, we compute:

- $n=10$
- $\sum x_i = 21$

So,

$$L(\lambda) = e^{-10\lambda} \prod_{i=1}^{10} \frac{1}{x_i!} \lambda^{x_i} L(\lambda) = e^{-10\lambda} \lambda^{21} \prod_{i=1}^{10} \frac{1}{x_i!}$$

The constant $\prod_{i=1}^{10} \frac{1}{x_i!}$ doesn't depend on λ , so it can be omitted for maximum likelihood or Bayesian posterior derivation.

```
# a) Given data in the question
X <- c(1, 3, 2, 2, 0, 3, 4, 3, 2, 1)
n <- length(X)
sum_x <- sum(X)
print(sum_x)
```

Output:

```
> print(sum_x)
[1] 21
```

(b) Prior Distribution

For Poisson distribution with parameter λ , the conjugate prior is the **Gamma distribution**:

$$\lambda \sim \text{Gamma}(\alpha, \beta)$$

$$\pi(\lambda) = \beta \alpha \Gamma(\alpha) \lambda^{\alpha-1} e^{-\beta \lambda} \quad \pi(\lambda) = \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda^{\alpha-1} e^{-\beta \lambda}$$

Let's choose optional hyperparameters:

Say $\alpha=2$, $\beta=1$ (This is a subjective choice unless specified; any positive values are acceptable.)

```
> # b) Prior hyperparameters
> alpha <- 2
> beta <- 1
> |
```

(c) Posterior Distribution

Bayes theorem:

$$\pi(\lambda|X) \propto L(\lambda) \cdot \pi(\lambda) \propto \pi(\lambda \mid X) \propto L(\lambda) \cdot \pi(\lambda)$$

Using:

- $L(\lambda) \propto e^{-10\lambda} \lambda^{21} L(\lambda) \propto e^{-10\lambda} \lambda^{21}$
- $\pi(\lambda) \propto \lambda^{\alpha-1} e^{-\beta\lambda} \pi(\lambda) \propto \lambda^{\alpha-1} e^{-\beta\lambda}$ So: $\pi(\lambda|X) \propto \lambda^{21+\alpha-1} e^{-(10+\beta)\lambda} \pi(\lambda \mid X) \propto \lambda^{21+\alpha-1} e^{-(10+\beta)\lambda}$

With $\alpha=2$, $\beta=1$:

$$\pi(\lambda|X) \propto \lambda^{22} e^{-11\lambda} \pi(\lambda \mid X) \propto \lambda^{22} e^{-11\lambda}$$

Thus, the posterior is:

$$\lambda|X \sim \text{Gamma}(23, 11) \quad \lambda|X \sim \text{Gamma}(23, 11)$$

```
# d) Posterior distribution is Gamma(alpha_post, beta_post)
posterior_mean <- alpha_post / beta_post

cat("Posterior: Gamma(", alpha_post, ",", beta_post, ")\n")
cat("Bayesian estimator (posterior mean):", posterior_mean, "\n")
```

(d) Bayesian Estimator

Under squared error loss, the Bayes estimator is the posterior mean:

$$\lambda^{\text{Bayes}} = E[\lambda|X] = \frac{\alpha + \sum x_i}{\beta + n} = \frac{23 + 11}{23 + 11} \approx 2.09$$

```
cat("Posterior: Gamma(", alpha_post, ",", beta_post, ")\n")
cat("Bayesian estimator (posterior mean):", posterior_mean, "\n")
```

Output:

```
> cat("Posterior: Gamma(", alpha_post, ",", beta_post, ")\n")
Posterior: Gamma( 23 , 11 )
> cat("Bayesian estimator (posterior mean):", posterior_mean, "\n")
Bayesian estimator (posterior mean): 2.090909
```

Question 3: Time Series Analysis of Stock Data

Load the necessary data

```
#install the needed packages
install.packages("tidyquant")
library(tidyquant)
library(tseries)
library(forecast)
library(quantmod)
```

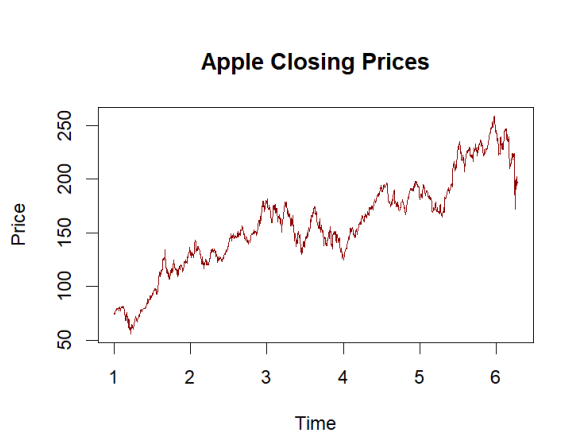
(a) Stationary Check

The analysis entry started from January 2020, studying the closing prices of stock issued by Apple Inc. (AAPL). For the sake of stationarity in mean and variance, differencing was applied, and then the series was log-transformed. The log_diff series was displayed to be more stationarity.

```
## (a) Check for stationarity
plot(aapl_ts, col = "darkred", main = "Apple Closing Prices", ylab = "Price")

# Apply differencing and log transformation
ts_diff <- diff(aapl_ts)
log_ts_diff <- diff(log(aapl_ts))
```

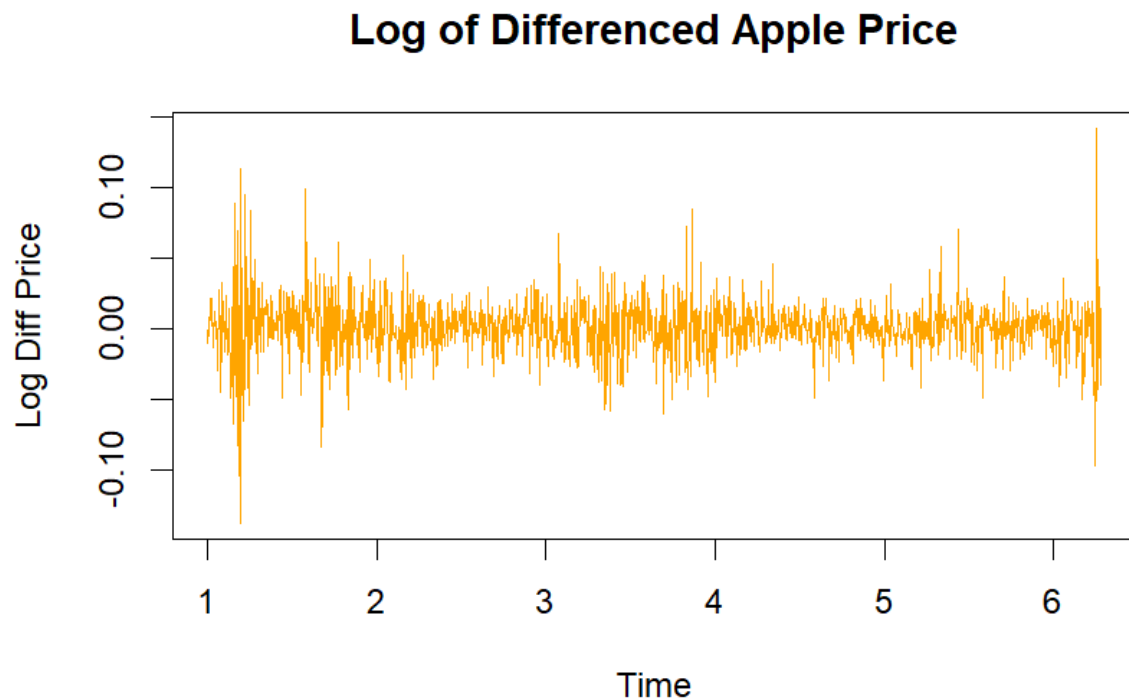
Plotting the apple closing prices:



Log difference:

```
# Plot log difference  
plot(log_ts_diff, col = "orange", main = "Log-Differenced Apple Price", ylab = "Log Difference")
```

The plot:

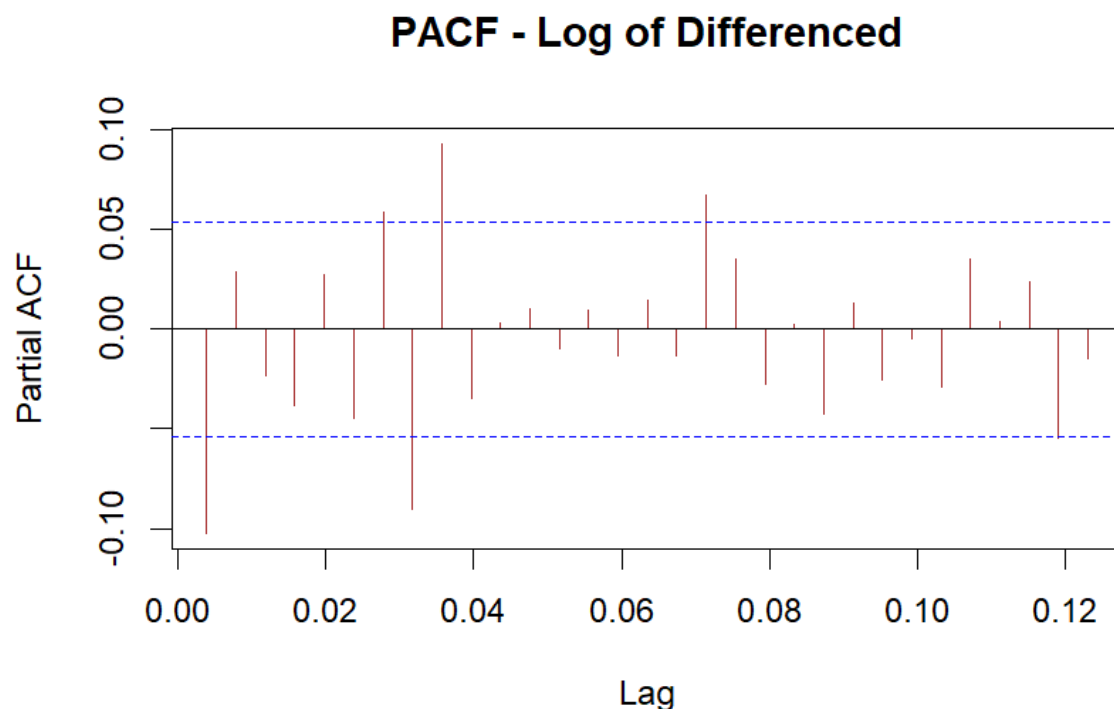


(b) ACF and PACF Analysis

ACF and PACF were plotted to identify AR and MA orders.

```
## (b) ACF and PACF plots  
acf(log_ts_diff, col = "navy", main = "ACF - Log Differenced")  
pacf(log_ts_diff, col = "brown", main = "PACF - Log Differenced")
```

The plot for Pacf plots



(c) Model Selection Using `auto.arima()`

`auto.arima()` was undertaken for automatically determining the best-fitting ARIMA model.

```
## (c) Auto ARIMA model selection
best_model <- auto.arima(log_ts_diff, seasonal = FALSE)
best_model
```

Picking the best model:

```
Series: log_ts_diff
ARIMA(1,0,0) with non-zero mean
```

Coefficients:

	ar1	mean
	-0.1021	7e-04
s.e.	0.0273	5e-04

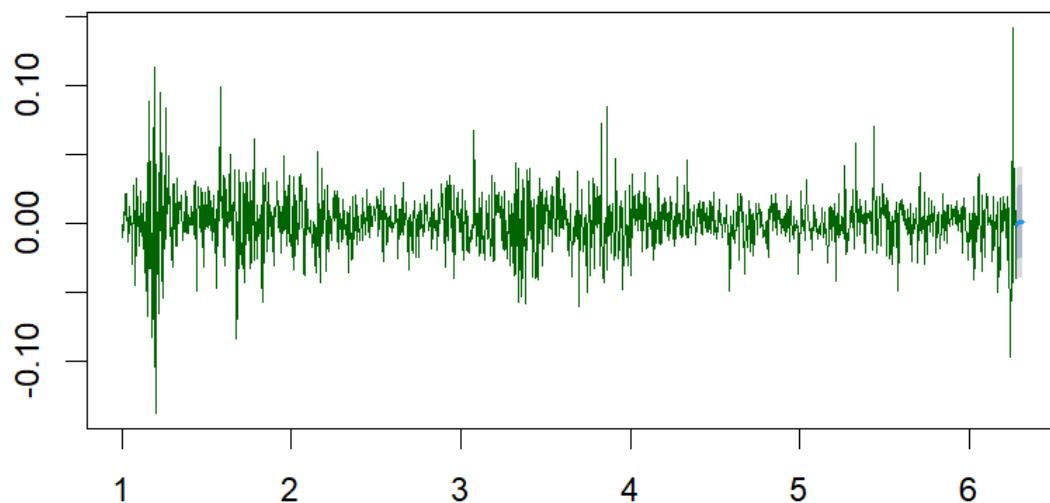
```
sigma^2 = 0.0004212: log likelihood = 3282.49
AIC=-6558.99   AICc=-6558.97   BIC=-6543.41
```

(d) Forecasting

A 10-step forecast using the selected ARIMA model was conducted with visualization.

```
## (d) Forecast 10 future steps  
forecast_10 <- forecast(best_model, h = 10)  
plot(forecast_10, col = "darkgreen", main = "10-Day Forecast of Apple Stock")
```

10-Day Forecast of Apple Stock



Conclusion

This report concerns analysis on GLM fitting, Bayesian estimation with Poisson data, and time Series forecasting through R.